Step 2 — The Graph: Answer or Escalate — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — three nodes, one decision
- Hint 2 — the escalate node is the interesting one

Hint 1 — three nodes, one decision

The graph is three nodes and a conditional edge:

```
retrieve_node -> fill context + best_distance (best_match)
route(state)  -> "answer" if is_confident(best_distance) else "escalate"
answer_node   -> ask the model to answer ONLY from state["context"]
escalate_node -> interrupt(...), then return the human reply
```

Wire it: `START -> retrieve`; `add_conditional_edges("retrieve", route, {"answer":"answer","escalate":"escalate"})`; `answer -> END`; `escalate -> END`. The `route` function is Step 1's gate — the whole decision, now visible in the graph.

Hint 2 — the escalate node is the interesting one

`answer_node` is a normal grounded RAG answer:

```
chain = prompt | model
reply = chain.invoke({"context": state["context"],
                      "question": state["question"]})
return {"answer": reply.content, "escalated": False}
```

`escalate_node` is where human-in-the-loop lives:

```
def escalate_node(state):
    human = interrupt({"question": state["question"]})   # PAUSES the graph
    return {"answer": human, "escalated": True}
```

`interrupt()` stops the whole graph and hands the question to whatever is driving it; its return value is the human's reply, which it only gets once the run resumes (Step 3). Have `build_graph`

return the *uncompiled* graph — Step 3 compiles it with the checkpointer that `interrupt` needs. The whole file set is in the Solution.